Python

▼ ***What is Python ?***

→ Python is a ***high-level, interpreted programming language*** known for its simplicity and readability. It is widely used for web development, data science, artificial intelligence, machine learning, automation, scripting, and more. Python emphasizes code readability with its clean syntax and indentation-based structure.

Key Features of Python:

- ***Interpreted*** : Executes code line by line.
- ***Dynamically Typed*** : No need to declare variable types.
- ***Extensive Libraries*** : Has rich libraries like NumPy, Pandas, TensorFlow, Django, Flask, etc.

▼ ***Installation of Python ?***

STEPS :

1. Go to the official Python website: https://www.python.org/downloads/
2. Click on **"Download Python [latest version]"** (e.g., Python 3.x.x).
3. The installer (`.exe` file) will be downloaded.
4. Open **Command Prompt (cmd)**:
5. Type `cmd`, and press **Enter**.
6. Type the following command and press **Enter** : If Python is installed correctly, it will display the installed version.

" python --version "

▼ ***What is Python Indentation ?***

→ Indentation in Python refers to ***the spaces or tabs at the beginning of a line of code*** that define the structure of the program. Unlike other programming languages that use curly braces `{}` to mark code blocks, Python **strictly enforces indentation** to indicate blocks of code.

→ The number of spaces is up to you as a programmer, the most common use is four, but it has to be at least one.

▼ ***Python Variables ?***

→ A **variable** in Python is used to store data. It acts as a container that holds values, which can be changed during program execution.

▼ ***Declaring a Variable in Python :***

- Python **automatically assigns** the data type based on the value.
- No need to specify `int`, `float`, `string`, etc.

```
x = 10  # Integer
y = "Sanjai"  # String
z = 3.14  # Float
```

▼ ***Variable Names :***

Rules for Naming Variables

✅ **Valid Variable Names**:

1. Must start with a **letter (a-z, A-Z) or an underscore** `_`, but not a number.

2. Can contain **letters, numbers (0-9), and underscores** `_`.
3. Variable names are **case-sensitive** (`age` and `Age` are different).
4. Should not use **Python keywords** (e.g., `if`, `else`, `class`, `def`, etc.).

```
name = "Sanjai"  # Valid
_age = 22  # Valid
totalAmount = 500  # Valid
```

Best Practices for Naming Variables

- Use **descriptive** names (`student_name` instead of `s`).
- Follow **snake_case** (`first_name`, `total_amount`).
- Use **camelCase** if working with JavaScript-like syntax (`firstName`).

▼ ***Assign Multiple Values :***

→ Python allows assigning **multiple values** to multiple variables in one line.

Example : ***Assign Different Values***

```
a, b, c = 10, 20, 30
print(a, b, c)  # Output: 10 20 30
```

Example : ***Assign Same Value to Multiple Variables***

```
x = y = z = 100
print(x, y, z)  # Output: 100 100 100
```

Example : ***Unpacking a List or Tuple***

```
fruits = ["apple", "banana", "cherry"]
x, y, z = fruits
print(x, y, z)  # Output: apple banana cherry
```

▼ ***Output Variables :***

→ You can display the value of variables using `print()`.

Example : ***Printing Variables***

```
name = "Sanjai"
age = 22
print(name)
print(age)
```

Example : ***Combining Strings and Variables***

```
name = "Sanjai"
print("Hello, " + name)  # Output: Hello, Sanjai
```

Example : ***Using*** `f-strings` ***(Recommended)***

```
age = 22
print(f"I am {age} years old.")  # Output: I am 22 years old.
```

Example : *Using* `format()`

```
name = "Sanjai"
age = 25
print("My name is {} and I am {} years old.".format(name, age))
```

▼ **Global Variables :**

→ A **global variable** is a variable declared outside of a function and can be accessed anywhere in the program.

Example : *Global Variable*

```
x = "Python"  # Global variable

def my_function():
  print("I love " + x)

my_function()  # Output: I love Python
```

Using the `global` ***Keyword***

→ To modify a **global variable** inside a function, use the `global` keyword.

```
x = "Python"  # Global variable

def my_function():
  global x
  x = "JavaScript"  # Modify global variable

my_function()
print(x)  # Output: JavaScript
```

▼ ***Python Data Types ?***

→ A **data type** defines the type of value a variable holds. Python provides several built-in data types.

▼ ***Types of Data Types :***

Text Type	`str`
Numeric Types	`int` , `float` , `complex`
Sequence Types	`list` , `tuple` , `range`
Mapping Type	`dict`
Set Types	`set` , `frozenset`
Boolean Type	`bool`
Binary Types	`bytes` , `bytearray` , `memoryview`
None Type	`NoneType`

▼ ***Setting the Specific Data Type :***

Example	Data Type	OutPut
x = str("Sanjai Kannan G")	**str**	**Sanjai Kannan G - str**
x = int(22)	**int**	**22 - int**
x = float(22.5)	**float**	**22.5 - float**
x = complex(1j)	**complex**	**1j - complex**
x = list(("sanjai", "kannan", "gokul"))	**list**	**["sanjai", "kannan", "gokul"] - list**

x = tuple(("sanjai", "kannan", "gokul"))	tuple	("sanjai", "kannan", "gokul") - tuple
x = range(6)	range	range(0, 6) - range
x = dist("name" : "sanjai", "age" : 22)	dict	{'name': 'sanjai', 'age': 22} - dist
x = set(("apple", "banana", "cherry"))	set	{'apple', 'banana', 'cherry'} - set
x = frozenset(("apple", "banana", "cherry"))	frozenset	frozenset({'banana', 'apple', 'cherry'})
x = bool(5)	bool	True
x = bytes(5)	bytes	b'\x00\x00\x00\x00\x00’
x = bytearray(5)	bytearray	bytearray(b'\x00\x00\x00\x00\x00')
x = memoryview(bytes(5))	memoryview	<memory at 0x00D58FA0>

▼ ***Python Casting (Type Conversion) ?***

→ Python allows you to convert data from one type to another using **casting functions**.

1. Casting to Integer (`int`) :

→ Converts values to an integer (removes decimals, converts strings if valid).

```
x = int(3.9)      # 3
y = int("10")     # 10
z = int(True)     # 1
```

2. Casting to Float (`float`) :

→ Converts values to a floating-point number.

```
x = float(5)      # 5.0
y = float("3.14")  # 3.14
z = float(False)   # 0.0
```

3. Casting to String (`str`) :

→ Converts values to a string.

```
x = str(100)     # "100"
y = str(3.14)    # "3.14"
z = str(True)    # "True"
```

4. Casting to Boolean (`bool`) :

→ Converts values to `True` or `False`.

- `0`, `None`, `""`, `[]`, `{}` → **False**
- Everything else → **True**

```
x = bool(0)       # False
y = bool(10)      # True
z = bool("Hello")  # True
```

5. Casting to List (`list`) :

→ Converts an iterable (tuple, string, range) to a list.

```
x = list("hello")      # ['h', 'e', 'l', 'l', 'o']
y = list((1, 2, 3))    # [1, 2, 3]
z = list(range(3))     # [0, 1, 2]
```

6. Casting to Tuple (`tuple`) :

→ Converts an iterable to a tuple.

```
x = tuple([1, 2, 3])    # (1, 2, 3)
y = tuple("hello")      # ('h', 'e', 'l', 'l', 'o')
```

7. Casting to Set (`set`) :

→ Converts an iterable to a set (removes duplicates).

```
python
CopyEdit
x = set([1, 2, 2, 3])   # {1, 2, 3}
y = set("banana")       # {'a', 'b', 'n'}
```

8. Casting to Dictionary (`dict`) :

→ Converts a list of key-value pairs to a dictionary.

```
x = dict([("name", "Sanjai"), ("age", 25)])
# {'name': 'Sanjai', 'age': 25}
```

▼ *Python Strings ?*

▼ *Strings :*

→ A **string** in Python is a sequence of characters enclosed in **single**, **double**, or **triple quotes**.

```
print("Sanjai") #Double Quotes
print('Sanjai')  #Single Quotes
```

▼ *Slicing Strings :*

→ Python allows **indexing** and **slicing** strings using `[]`.

```
text = "Sanjai Kannan G"

# Accessing characters
print(text[0])    # S (first character)
print(text[-1])   # G (last character)

# Slicing [start:end:step]
print(text[0:6])   # Python (0 to 5)
print(text[7:])    # Kannan G (from index 7 to end)
print(text[:6])    # Sanja (start to index 5)
print(text[::2])   # Sna annG (every second character)
print(text[::-1])  # G nannaK iajnaS (Reverse string)
```

▼ *Modify Strings :*

→ Python provides multiple string modification methods.

```
text = " Sanjai Kannan G"

# Changing case
print(text.upper())    # " HELLO WORLD "
print(text.lower())    # " hello world "
print(text.title())    # " Hello World "
print(text.capitalize()) # " Hello world "

# Removing spaces
print(text.strip())   # "hello world"
print(text.lstrip())  # "hello world " (removes left spaces)
print(text.rstrip())  # " hello world" (removes right spaces)

# Replacing
print(text.replace("hello", "Hi"))  # " Hi world "
```

▼ ***Concatenate Strings***

→ In Python, you can concatenate strings using the `+` operator or using the `join()` method.

```
# USING " + "

str1 = "sanjai"
str2 = "kannan"

result = str1 + " " + str2   # Concatenate with a space
print(result)  # Output: sanjai kannan
```

```
# USING " JOIN "

str1 = "sanjai"
str2 = "kannan"

result = " ".join([str1, str2])  # Join with a space
print(result)  # Output: sanjai kannan
```

▼ ***Format Strings***

→ In 2 Methods we can format a string in python.

1. **Using f-strings** (Python 3.6+)
2. **Using** `.format()`

```
# Using f-strings

str1 = "sanjai"

result = f"my name is {str1}"
print(result)
```

```
# Using .format()

name = "sanjai"
age = 22

result = "my name is {} and my age is {}".format(name, age)
print(result)
```

▼ ***Escape Characters***

→ Escape characters allow you to insert special characters into strings. For example:

- `\n`: Newline
- `\t`: Tab
- `\\`: Backslash
- `\'`: Single quote
- `\"`: Double quote

```
str1 = "Sanjai\nKannan"  # Newline between Sanjai and Kannan
print(str1)  # Output:
# Sanjai
# Kannan

str2 = "This is a \"quoted\" word."
print(str2)  # Output: This is a "quoted" word.

str3 = "Backslash: \\"
print(str3)  # Output: Backslash: \
```

▼ ***String Methods***

PYTHON STRING METHODS

No	Method	Description	Example
1	capitalize()	Converts the first character to upper case	`"hello".capitalize()` → `'Hello'`
2	casefold()	Converts string into lower case	`"HELLO".casefold()` → `'hello'`
3	center()	Returns a centered string	`"hello".center(10, "*")` → `'**hello***'`
4	count()	Returns the number of times a specified value occurs	`"hello".count('l')` → `2`
5	encode()	Returns an encoded version of the string	`"hello".encode()` → `b'hello'`
6	endswith()	Returns true if the string ends with the specified value	`"hello".endswith('o')` → `True`
7	expandtabs()	Sets the tab size of the string	`"hello\tworld".expandtabs(4)` → `'hello world'`
8	find()	Searches the string for a specified value	`"hello".find('e')` → `1`

No	Method	Description	Example
9	format()	Formats specified values in a string	`"My name is {}".format("Alice")` → `'My name is Alice'`
10	format_map()	Formats specified values in a string	`"My name is {name}".format_map({"name": "Alice"})` → `'My name is Alice'`
11	index()	Searches the string for a specified value	`"hello".index('e')` → `1`
12	isalnum()	Returns True if all characters are alphanumeric	`"hello123".isalnum()` → `True`
13	isalpha()	Returns True if all characters are in the alphabet	`"hello".isalpha()` → `True`
14	isascii()	Returns True if all characters are ascii characters	`"hello".isascii()` → `True`
15	isdecimal()	Returns True if all characters are decimals	`"123".isdecimal()` → `True`
16	isdigit()	Returns True if all characters are digits	`"123".isdigit()` → `True`
17	isidentifier()	Returns True if the string is an identifier	`"my_var".isidentifier()` → `True`
18	islower()	Returns True if all characters are lower case	`"hello".islower()` → `True`
29	isnumeric()	Returns True if all characters are numeric	`"123".isnumeric()` → `True`
20	isprintable()	Returns True if all characters in the string are printable	`"hello".isprintable()` → `True`
21	isspace()	Returns True if all characters are whitespaces	`" ".isspace()` → `True`
22	istitle()	Returns True if the string follows the rules of a title	`"Hello World".istitle()` → `True`
23	isupper()	Returns True if all characters are upper case	`"HELLO".isupper()` → `True`
24	join()	Joins the elements of an iterable to the end of the string	`"-".join(['hello', 'world'])` → `'hello-world'`
25	ljust()	Returns a left justified version of the string	`"hello".ljust(10, '*')` → `'hello*****'`
26	lower()	Converts a string into lower case	`"HELLO".lower()` → `'hello'`
27	lstrip()	Returns a left trim version of the string	`" hello".lstrip()` → `'hello'`
28	maketrans()	Returns a translation table to be used in translations	`"abc".maketrans("a", "z")` → `translation table`
29	partition()	Returns a tuple where the string is parted into three parts	`"hello world".partition(" ")` → `('hello', ' ', 'world')`
30	replace()	Returns a string where a specified value is replaced	`"hello".replace("e", "a")` → `'hallo'`
31	rfind()	Searches the string for a specified value (last occurrence)	`"hello".rfind('l')` → `3`
32	rindex()	Searches the string for a specified value (last occurrence)	`"hello".rindex('l')` → `3`
33	rjust()	Returns a right justified version of the string	`"hello".rjust(10, '*')` → `'*****hello'`
34	rpartition()	Returns a tuple where the string is parted into three parts (from right)	`"hello world".rpartition(" ")` → `('hello', ' ', 'world')`
35	rsplit()	Splits the string at the specified separator, returns a list	`"a,b,c".rsplit(',')` → `['a', 'b', 'c']`

No	Method	Description	Example
36	rstrip()	Returns a right trim version of the string	`"hello ".rstrip()` → `'hello'`
37	split()	Splits the string at the specified separator, returns a list	`"a,b,c".split(',')` → `['a', 'b', 'c']`
38	splitlines()	Splits the string at line breaks and returns a list	`"hello\nworld".splitlines()` → `['hello', 'world']`
39	startswith()	Returns true if the string starts with the specified value	`"hello".startswith('h')` → `True`
40	strip()	Returns a trimmed version of the string	`" hello ".strip()` → `'hello'`
41	swapcase()	Swaps cases, lower case becomes upper case and vice versa	`"Hello".swapcase()` → `'hELLO'`
42	title()	Converts the first character of each word to upper case	`"hello world".title()` → `'Hello World'`
43	translate()	Returns a translated string	`"abc".translate({97: 65})` → `'Abc'`
44	upper()	Converts a string into upper case	`"hello".upper()` → `'HELLO'`
45	zfill()	Fills the string with a specified number of 0 values at the beginning	`"42".zfill(5)` → `'00042'`

▼ ***Python Booleans ?***

Boolean Values :

→ In programming you often need to know if an expression is `True` or `False`.

→ You can evaluate any expression in Python, and get one of two answers, `True` or `False`.

→ When you compare two values, the expression is evaluated and Python returns the Boolean answer:

```
print(10 > 9)
print(10 == 9)
print(10 < 9)

# Output
True
False
False
```

Evaluate Values and Variables :

→ The `bool()` function allows you to evaluate any value, and give you `True` or `False` in return,

```
print(bool("Hello"))
print(bool(15))

True
True
```

Most Values are True :

→ Almost any value is evaluated to `True` if it has some sort of content.

→ Any string is `True`, except empty strings.

→ Any number is `True`, except `0`.

→ Any list, tuple, set, and dictionary are `True`, except empty ones.

```
print(bool("abc"))
print(bool(123))
print(bool(["apple", "cherry", "banana"]))

# Output
True
True
True
```

Some Values are False :

→ In fact, there are not many values that evaluate to `False`, except empty values, such as `()`, `[]`, `{}`, `""`, the number `0`, and the value `None`. And of course the value `False` evaluates to `False`.

```
print(bool(False))
print(bool(None))
print(bool(0))
print(bool(""))
print(bool(()))
print(bool([]))
print(bool({}))

# Output
False
False
False
False
False
False
False
```

▼ ***Python Operators ?***

→ Operators are used to perform operations on variables and values.

Python divides the operators in the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators
- Membership operators
- Bitwise operators

Python Arithmetic Operators

→ Arithmetic operators are used with numeric values to perform common mathematical operations:

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
%	Modulus	x % y
**	Exponentiation	x ** y
//	Floor division	x // y

Python Assignment Operators

→ Assignment operators are used to assign values to variables:

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
`	=`	`x
^=	x ^= 3	x = x ^ 3
>≥	x >>= 3	x = x >> 3
<≤	x <<= 3	x = x << 3
:=	x := 3	x = 3

Python Comparison Operators

→ Comparison operators are used to compare two values:

Operator	Name	Example
==	Equal	x == y
≠	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
≥	Greater than or equal to	x >= y
≤	Less than or equal to	x <= y

Python Logical Operators

→ Logical operators are used to combine conditional statements:

Operator	**Description**	**Example**
and	Returns “True” if both statements are true	x < 5 and x < 10
or	Returns “True” if at least one of the statements is true	x < 5 or x < 4
not	Reverses the result, returns “False” if the result is true	not(x < 5 and x < 10)

Python Identity Operators

→ Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

Operator	Description	Example
is	Returns “True” if both variables point to the same object	x is y
is not	Returns “True” if both variables do not point to the same object	x is not y

Python Membership Operators

→ Membership operators are used to test if a sequence is presented in an object:

Operator	Description	Example
in	Returns “True” if a sequence with the specified value is present in the object	x in y
not in	Returns “True”if a sequence with the specified value is not present in the object	x not in y

Python Bitwise Operators

→ Bitwise operators are used to compare (binary) numbers:

Operator	Name	Description	Example
&	AND	Sets each bit to `1` if both bits are `1`	`x & y`
|	OR	Sets each bit to 1 if one of two bits is 1	x | y
^	XOR	Sets each bit to 1 if only one of two bits is 1	x ^ y
~	NOT	Inverts all the bits	~x
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off	x << 2
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off	x >> 2

▼ *Python Lists ?*

List :

→ Lists are used to store multiple items in a single variable. Lists are created using square brackets:

```
# Syntax
names = ["sanjai", "kannan", "gokul"]
print(names)

# Output
['sanjai', 'kannan', 'gokul']
```

List Items :

→ List items are ordered, changeable, and allow duplicate values. List items are indexed, the first item has index `[0]`, the second item has index `[1]` etc.

Ordered :

→ When we say that lists are ordered, it means that the items have a defined order, and that order will not change.

→ If you add new items to a list, the new items will be placed at the end of the list.

Changeable :

→ The list is changeable, meaning that we can change, add, and remove items in a list after it has been created.

Allow Duplicates :

→ Since lists are indexed, lists can have items with the same value.

▼ ***Access List Items :***

Access Items :

→ List items are indexed and you can access them by referring to the index number:

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
print(names[1])

# Output
kannan
```

Negative Indexing :

→ Negative indexing means start from the end

→ `1` refers to the last item, `2` refers to the second last item etc.

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
print(names[-1])

# Output
suja
```

Range of Indexes :

→ You can specify a range of indexes by specifying where to start and where to end the range.

→ When specifying a range, the return value will be a new list with the specified items.

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
print(names[1:4])

# Output
['kannan', 'gokul', 'Kannan']
```

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
print(names[:2])

# Output
['sanjai', 'kannan']
```

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
print(names[2:])

# Output
['gokul', 'Kannan', 'guna', 'suja']
```

Range of Negative Indexes :

→ Specify negative indexes if you want to start the search from the end of the list:

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
print(names[-4:-1])

# Output
['gokul', 'Kannan', 'guna']
```

Check if Item Exists :

→ To determine if a specified item is present in a list use the `in` keyword:

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
if "gokul" in names:
   print("yes")
else:
   print("no")

# Output
yes
```

▼ ***Change List Items :***

Change Value :

→ To change the value of a specific item, refer to the index number:

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
names[1] = "kannanChanged"
print(names)

# Output
['sanjai', 'kannanChanged', 'gokul', 'Kannan', 'guna', 'suja']
```

Change a Range of Item Values :

→ To change the value of items within a specific range, define a list with the new values, and refer to the range of index numbers where you want to insert the new values:

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
names[1:3] = ["kannanChanged", "gokulChanged"]
print(names)

# Output
['sanjai', 'kannanChanged', 'gokulChanged', 'Kannan', 'guna', 'suja']
```

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
names[1] = ["kannanChanged", "kannanChanged"]
print(names)

# Output
['sanjai', ['kannanChanged', 'kannanChanged'], 'gokul', 'Kannan', 'guna', 'suja']
```

Insert Items :

→ To insert a new list item, without replacing any of the existing values, we can use the `insert()` method.

→ The `insert()` method inserts an item at the specified index:

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
names.insert(2, "kannanNameChangedToSK")
print(names)

# Output
['sanjai', 'kannan', 'kannanNameChangedToSK', 'gokul', 'Kannan', 'guna', 'suja']
```

▼ ***Add List Items :***

Append Items :

→ To add an item to the end of the list, use the append() method:

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
names.append("appendedName")
print(names)

# Output
['sanjai', 'kannan', 'gokul', 'Kannan', 'guna', 'suja', 'appendedName']
```

Insert Items :

→ To insert a list item at a specified index, use the `insert()` method.

→ The `insert()` method inserts an item at the specified index:

```
names = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
names.insert(1, "InsertedName")
print(names)

# Output
['sanjai', 'InsertedName', 'kannan', 'gokul', 'Kannan', 'guna', 'suja']
```

Extend List :

→ o append elements from *another list* to the current list, use the `extend()` method.

```
first_name_list = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
second_name_list = ["Ravi", "John", "Doe"]
first_name_list.extend(second_name_list)
print(first_name_list)

# Output
['sanjai', 'kannan', 'gokul', 'Kannan', 'guna', 'suja', 'Ravi', 'John', 'Doe']
```

Add Any Iterable :

→ The `extend()` method does not have to append *lists*, you can add any iterable object (tuples, sets, dictionaries etc.).

```
first_name_list = ["sanjai", "kannan", "gokul", "Kannan", "guna", "suja"]
second_name_list = ("Ravi", "John", "Doe")
first_name_list.extend(second_name_list)
print(first_name_list)

# Output
['sanjai', 'kannan', 'gokul', 'Kannan', 'guna', 'suja', 'Ravi', 'John', 'Doe']
```

▼ ***Remove List Items :***

Remove Specified Item:

→ The `remove()` method removes the specified item.

```
names = [ "sanjai", "kannan", "gokul" ]
names.remove("sanjai")
print(names)

# Output
['kannan', 'gokul']
```

Remove Specified Index :

→ The `pop()` method removes the specified index.

```
names = [ "sanjai", "kannan", "gokul" ]
names.pop()          # Remove the last item
print(names)

# Output
['sanjai', 'kannan']
```

```
names = [ "sanjai", "kannan", "gokul" ]
names.pop(1)          # Remove the second item
print(names)
```

```
# Output
['sanjai', 'gokul']
```

→ The `del` keyword also removes the specified index:

```
names = [ "sanjai", "kannan", "gokul" ]
del names[0]
print(names)

# Output
['kannan', 'gokul']
```

→ The `clear()` method empties the list. The list still remains, but it has no content.

```
names = [ "sanjai", "kannan", "gokul" ]
names.clear()
print(names)

# Output
[]
```

▼ ***Loop List :***

Loop Through a List :

→ You can loop through the list items by using a `for` loop:

```
names = [ "sanjai", "kannan", "gokul" ]
for x in names:
    print(x)

# Output
sanjai
kannan
gokul
```

Loop Through the Index Numbers :

→ You can also loop through the list items by referring to their index number.

→ Use the `range()` and `len()` functions to create a suitable iterable

```
names = [ "sanjai", "kannan", "gokul" ]
for x in range(len(names)):
    print(x)

# Output
```

```
0
1
2
```

Using a While Loop :

→ You can loop through the list items by using a `while` loop.

→ Use the `len()` function to determine the length of the list, then start at 0 and loop your way through the list items by referring to their indexes.

→ Remember to increase the index by 1 after each iteration.

```
names = [ "sanjai", "kannan", "gokul" ]
i = 0
while i < len(names):
  print(names[i])
  i = i + 1

# Output
sanjai
kannan
gokul
```

▼ ***Sort List :***

Sort List Alphanumerically :

→ List objects have a `sort()` method that will sort the list alphanumerically, ascending, by default:

```
names = ["sanjai", "aparna", "bala", "praveen"]
names.sort()   # Sort the list alphabetically
print(names)

# Output
['aparna', 'bala', 'praveen', 'sanjai']
```

```
names = [100, 10, 200, 2, 9]
names.sort()     # Sort the list numerically
print(names)

# Output
[2, 9, 10, 100, 200]
```

Sort Descending :

→ To sort descending, use the keyword argument `reverse = True`:

```
names = ["sanjai", "aparna", "bala", "praveen"]
names.sort(reverse = True)    # Sort the list descending
print(names)

# Output
['sanjai', 'praveen', 'bala', 'aparna']
```

```
names = [100, 10, 200, 2, 9]
names.sort(reverse = True)     # Sort the list descending
print(names)

# Output
[200, 100, 10, 9, 2]
```

Reverse Order :

→ What if you want to reverse the order of a list, regardless of the alphabet?

→ The `reverse()` method reverses the current sorting order of the elements.

```
names = ["sanjai", "aparna", "bala", "praveen"]
names.reverse()    # Reverse the order of the list
print(names)

# Output
['praveen', 'bala', 'aparna', 'sanjai']
```

▼ ***Copy List :***

Copy a List :

→ You cannot copy a list simply by typing `list2 = list1`, because: `list2` will only be a *reference* to `list1`, and changes made in `list1` will automatically also be made in `list2`.

Use the copy() method :

→ You can use the built-in List method `copy()` to copy a list.

```
names = ["sanjai", "aparna", "bala", "praveen"]
new_names = names.copy()
print(new_names)

# Output
['sanjai', 'aparna', 'bala', 'praveen']
```

```
names = ["sanjai", "aparna", "bala", "praveen"]
new_names = list(names)
print(new_names)


# Output
['sanjai', 'aparna', 'bala', 'praveen']
```

```
names = ["sanjai", "aparna", "bala", "praveen"]
new_names = names[:]
print(new_names)


# Output
['sanjai', 'aparna', 'bala', 'praveen']
```

▼ ***Join List :***

Join Two Lists :

→ There are several ways to join, or concatenate, two or more lists in Python.

→ One of the easiest ways are by using the `+` operator.

```
names1 = ["sanjai", "aparna", "bala", "praveen"]
names2 = [ 1, 2, 3, 4, 5 ]
new_names = names1 + names2
print(new_names)

# Output
['sanjai', 'aparna', 'bala', 'praveen', 1, 2, 3, 4, 5]
```

```
names1 = ["sanjai", "aparna", "bala", "praveen"]
names2 = [ 1, 2, 3, 4, 5 ]
names1.extend(names2)
print(names1)
```

```
# Output
['sanjai', 'aparna', 'bala', 'praveen', 1, 2, 3, 4, 5]
```

▼ ***List Methods :***

List Methods :

Method	Description	Example
append()	Adds an element at the end of the list	`lst = [1, 2, 3]lst.append(4)` → `[1, 2, 3, 4]`
clear()	Removes all the elements from the list	`lst = [1, 2, 3]lst.clear()` → `[]`
copy()	Returns a copy of the list	`lst = [1, 2, 3]lst_copy = lst.copy()` → `[1, 2, 3]`
count()	Returns the number of elements with the specified value	`lst = [1, 2, 2, 3]lst.count(2)` → `2`
extend()	Adds the elements of a list (or any iterable) to the end	`lst = [1, 2]lst.extend([3, 4])` → `[1, 2, 3, 4]`
index()	Returns the index of the first element with the specified value	`lst = [1, 2, 3]lst.index(2)` → `1`
insert()	Adds an element at the specified position	`lst = [1, 2, 3]lst.insert(1, 4)` → `[1, 4, 2, 3]`
pop()	Removes the element at the specified position	`lst = [1, 2, 3]lst.pop(1)` → `2` New list: `[1, 3]`
remove()	Removes the item with the specified value	`lst = [1, 2, 2, 3]lst.remove(2)` → `[1, 2, 3]`
reverse()	Reverses the order of the list	`lst = [1, 2, 3]lst.reverse()` → `[3, 2, 1]`
sort()	Sorts the list	`lst = [3, 1, 2]lst.sort()` → `[1, 2, 3]`

▼ ***Python Tuples ?***

Tuple :

→ Tuples are used to store multiple items in a single variable.

→ A tuple is a collection which is ordered and unchangeable.

→ Tuples are written with round brackets.

```
names = ("sanjai","kannan","gokul")
print(names)

# Output
('sanjai', 'kannan', 'gokul')
```

Tuple Items :

→ Tuple items are ordered, unchangeable, and allow duplicate values.

→ Tuple items are indexed, the first item has index `[0]`, the second item has index `[1]` etc.

Ordered :

→ When we say that tuples are ordered, it means that the items have a defined order, and that order will not change.

Unchangeable :

→ Tuples are unchangeable, meaning that we cannot change, add or remove items after the tuple has been created.

**********IMPORTANT**********

Python Collections (Arrays)

There are four collection data types in the Python programming language:

- **List** is a collection which is ordered and changeable. Allows duplicate members.
- Tuple is a collection which is ordered and unchangeable. Allows duplicate members.
- **Set** is a collection which is unordered, unchangeable*, and unindexed. No duplicate members.
- **Dictionary** is a collection which is ordered** and changeable. No duplicate members.

▼ ***Access Tuples :***

Access Tuple Items :

→ You can access tuple items by referring to the index number, inside square brackets:

```
names = ("sanjai","kannan","gokul", "guna", "suja")
print(names[1])

# Output
kannan
```

Negative Indexing :

Negative indexing means start from the end.

- `-1` refers to the last item, `-2` refers to the second last item etc.

```
names = ("sanjai","kannan","gokul", "guna", "suja")
print(names[-1])

# Output
suja
```

Range of Indexes :

→ You can specify a range of indexes by specifying where to start and where to end the range.

→ When specifying a range, the return value will be a new tuple with the specified items.

```
names = ("sanjai","kannan","gokul", "guna", "suja")
print(names[1:3])

# Output
('kannan', 'gokul')
```

```
names = ("sanjai","kannan","gokul", "guna", "suja")
print(names[:3])

# Output
('sanjai', 'kannan', 'gokul')
```

```
names = ("sanjai","kannan","gokul", "guna", "suja")
print(names[3:])

# Output
('guna', 'suja')
```

Range of Negative Indexes :

→ Specify negative indexes if you want to start the search from the end of the tuple:

```
names = ("sanjai","kannan","gokul", "guna", "suja")
print(names[-3:-1])

# Output
('gokul', 'guna')
```

▼ ***Update Tuples :***

→ Tuples are unchangeable, meaning that you cannot change, add, or remove items once the tuple is created.

Change Tuple Values :

→ Once a tuple is created, you cannot change its values. Tuples are **unchangeable**, or **immutable** as it also is called.

→ But, You can convert the tuple into a list, change the list, and convert the list back into a tuple.

```
names_tuple = ("sanjai","kannan","gokul", "guna", "suja")
names_list = list(names_tuple)
print(names_list)

names_list[1] = "Newly Added name"
names_tuple = tuple(names_list)
print(names_tuple)
```

```
# Output
['sanjai', 'kannan', 'gokul', 'guna', 'suja']
('sanjai', 'Newly Added name', 'gokul', 'guna', 'suja')
```

Add Items :

→ Since tuples are immutable, they do not have a built-in `append()` method, but there are other ways to add items to a tuple.

1. **Convert into a list**: Just like the workaround for *changing* a tuple, you can convert it into a list, add your item(s), and convert it back into a tuple.

```
names_tuple = ("sanjai","kannan","gokul", "guna", "suja")
names_list = list(names_tuple)
names_list.append("newly added name")
names_tuple = tuple(names_list)
print(names_tuple)

# Output
('sanjai', 'kannan', 'gokul', 'guna', 'suja', 'newly added name')
```

2. **Add tuple to a tuple**. You are allowed to add tuples to tuples, so if you want to add one item, (or many), create a new tuple with the item(s), and add it to the existing tuple:

```
names_tuple1 = ("sanjai","kannan","gokul", "guna", "suja")
names_tuple2 = ("sanjai","kannan","gokul", "guna", "suja")
result = names_tuple1 + names_tuple2
print(result)

# Output
('sanjai', 'kannan', 'gokul', 'guna', 'suja', 'sanjai', 'kannan', 'gokul', 'guna', 'suja')
```

Remove Items :

→ Tuples are **unchangeable**, so you cannot remove items from it, but you can use the same workaround as we used for changing and adding tuple items:

```
names_tuple = ("sanjai","kannan","gokul", "guna", "suja")
names_list = list(names_tuple)
names_list.remove("sanjai")
names_tuple = tuple(names_list)
print(names_tuple)

# Output
('kannan', 'gokul', 'guna', 'suja')
```

```
names_tuple = ("sanjai","kannan","gokul", "guna", "suja")
del names_tuple
print(names_tuple)
```

```
# Output
error because the tuple no longer exists
```

▼ ***Unpack Tuple :***

→ When we create a tuple, we normally assign values to it. This is called "packing" a tuple.

→ But, in Python, we are also allowed to extract the values back into variables. This is called "unpacking".

```
names = ("sanjai", "kannan", "gokul")
(red, yellow, green) = names

print(red)
print(yellow)
print(green)

# Output
sanjai
kannan
gokul
```

Using Asterisk :

→ If the number of variables is less than the number of values, you can add an `*` to the variable name and the values will be assigned to the variable as a **list**.

```
names = ("sanjai", "kannan", "gokul", "kannan", "raja")
(red, yellow, *green) = names

print(red)
print(yellow)
print(green)

# Output
sanjai
```

```
kannan
['gokul', 'kannan', 'raja']
```

▼ ***Loop Tuple :***

→ You can loop through the tuple items by using a `for` loop.

```
names = ("sanjai", "kannan", "gokul")
for x in names:
   print(x)

# Output
sanjai
kannan
gokul
```

Loop Through the Index Numbers :

→ You can also loop through the tuple items by referring to their index number.

→ Use the `range()` and `len()` functions to create a suitable iterable.

```
names = ("sanjai", "kannan", "gokul")
for x in range(len(names)):
   print(names[x])

# Output
sanjai
kannan
gokul
```

▼ ***Tuple Methods :***

→ Python has two built-in methods that you can use on tuples.

Method	Description	Example
count()	Returns the number of times a specified value occurs in a tuple	my_tuple = (1, 2, 2, 3, 2) print(my_tuple.count(2)) # Output: 3

Method	Description	Example
index()	Searches the tuple for a specified value and returns the position of where it was found	my_tuple = (1, 2, 3, 4, 5) print(my_tuple.index(3)) # Output: 2

▼ ***Python Sets ?***

Set :

→ Sets are used to store multiple items in a single variable.

→ A set is a collection which is unordered, unchangeable*, and unindexed.

```
names = {"sanjai", "kannan", "gokul", "kannan"}
print(names)
print(type(names))

# Output
{'sanjai', 'gokul', 'kannan'}  # this do not allow duplicate values
<class 'set'>
```

Set Items :

→ Set items are **unordered, unchangeable, and do not allow duplicate values**.

Unordered :

→ Unordered means that the items in a set do not have a defined order.

→ Set items can appear in a different order every time you use them, and cannot be referred to by index or key.

Unchangeable :

→ Set items are unchangeable, meaning that we cannot change the items after the set has been created.

▼ ***Access Items :***

→ You cannot access items in a set by referring to an index or a key.

→ But you can loop through the set items using a `for` loop, or ask if a specified value is present in a set, by using the `in` keyword.

```
fruits = {"apple", "banana", "cherry"}

for x in fruits:
  print(x)
```

```
# Output
apple
banana
cherry
```

▼ ***Add Items :***

Once a set is created, you cannot change its items, but you can add new items.

→ To add one item to a set use the `add()` method.

```
fruits = {"apple", "banana", "cherry"}
fruits.add("orange")
print(fruits)

# Output
{'apple', 'cherry', 'orange', 'banana'}
```

Add Sets :

→ To add items from another set into the current set, use the `update()` method.

```
fruits = {"apple", "banana", "cherry"}
new_fruits = {"orange", "strawbery"}

fruits.update(new_fruits)
print(fruits)

# Output
{'apple', 'cherry', 'strawbery', 'orange', 'banana'}
```

▼ ***Remove Item :***

→ To remove an item in a set, use the `remove()`, or the `discard()` method. and clear()

```
fruits = {"apple", "banana", "cherry"}

fruits.remove("apple")
print(fruits)

# Output
{'banana', 'cherry'}
```

```
fruits = {"apple", "banana", "cherry"}

fruits.discard("apple")
print(fruits)
```

```
# Output
{'banana', 'cherry'}
```

```
fruits = {"apple", "banana", "cherry"}

fruits.clear()
print(fruits)

# Output
set()
```

▼ ***Join Sets :***

→ The `union()` and `update()` methods joins all items from both sets.

→ The `intersection()` method keeps ONLY the duplicates.

→ The `difference()` method keeps the items from the first set that are not in the other set(s).

→ The `symmetric_difference()` method keeps all items EXCEPT the duplicates.

```
fruits1 = {"apple", "banana", "cherry"}
fruits2 = {"orange", "strawbery"}
fruits3 = {"pomogranet"}

result = fruits1.union(fruits2, fruits3)   # using union()
print(result)

# Output
{'pomogranet', 'apple', 'banana', 'cherry', 'orange', 'strawbery'}
```

```
fruits1 = {"apple", "banana", "cherry"}
fruits2 = {"orange", "strawbery"}
fruits3 = {"pomogranet"}

result = fruits1 | fruits2 | fruits3   # using |
print(result)

# Output
{'pomogranet', 'apple', 'banana', 'cherry', 'orange', 'strawbery'}
```

```
fruits1 = {"apple", "banana", "cherry"}
fruits2 = {"orange", "apple"}

result = fruits1.intersection(fruits2)  # using intersection()
print(result)

# Output
{'apple'}
```

```
fruits1 = {"apple", "banana", "cherry"}
fruits2 = {"orange", "apple"}

result = fruits1 & fruits2
# using &
print(result)

# Output
{'apple'}
```

```
fruits1 = {"apple", "banana", "cherry"}
fruits2 = {"orange", "apple"}

result = fruits1.difference(fruits2)     # using difference()
print(result)

#Output
{'banana', 'cherry'}
```

```
fruits1 = {"apple", "banana", "cherry"}
fruits2 = {"orange", "apple"}

result = fruits1 - fruits2     # using -
print(result)

#Output
{'banana', 'cherry'}
```

```
fruits1 = {"apple", "banana", "cherry"}
fruits2 = {"orange", "apple"}

result = fruits1.symmetric_difference(fruits2)   # using symmetric_difference()
print(result)

#Output
{'cherry', 'banana', 'orange'}
```

```
fruits1 = {"apple", "banana", "cherry"}
fruits2 = {"orange", "apple"}

result = fruits1 ^ fruits2   # using ^
print(result)

#Output
{'cherry', 'banana', 'orange'}
```

▼ ***Set Methods :***

→ Python has a set of built-in methods that you can use on sets.

Method	Shortcut	Description	Example
add()		Adds an element to the set	`my_set = {1, 2}; my_set.add(3); print(my_set)` **Output:** `{1, 2, 3}`
clear()		Removes all the elements from the set	`my_set = {1, 2, 3}; my_set.clear(); print(my_set)` **Output:** `set()`
copy()		Returns a copy of the set	`my_set = {1, 2, 3}; new_set = my_set.copy(); print(new_set)` **Output:** `{1, 2, 3}`
difference()		Returns a set containing the difference between two or more sets	`set1 = {1, 2, 3}; set2 = {2, 3, 4}; print(set1.difference(set2))` **Output:** `{1}`
difference_update()	-=	Removes the items in this set that are also included in another, specified set	`set1 = {1, 2, 3}; set2 = {2, 3, 4}; set1.difference_update(set2); print(set1)` **Output:** `{1}`
discard()		Removes the specified item	`my_set = {1, 2, 3}; my_set.discard(2); print(my_set)` **Output:** `{1, 3}`
intersection()	&	Returns a set that is the intersection of two other sets	`set1 = {1, 2, 3}; set2 = {2, 3, 4}; print(set1.intersection(set2))` **Output:** `{2, 3}`
intersection_update()	&=	Removes the items in this set that are not present in other, specified set(s)	`set1 = {1, 2, 3}; set2 = {2, 3, 4}; set1.intersection_update(set2); print(set1)` **Output:** `{2, 3}`
isdisjoint()		Returns whether two sets have an intersection or not	`set1 = {1, 2, 3}; set2 = {4, 5, 6}; print(set1.isdisjoint(set2))` **Output:** `True`
issubset()	≤	Returns whether another set contains this set or not	`set1 = {1, 2}; set2 = {1, 2, 3}; print(set1.issubset(set2))` **Output:** `True`
<		Returns whether all items in this set are present in other, specified set(s)	`set1 = {1, 2}; set2 = {1, 2, 3}; print(set1 < set2)` **Output:** `True`
issuperset()	≥	Returns whether this set contains another set or not	`set1 = {1, 2, 3}; set2 = {1, 2}; print(set1.issuperset(set2))` **Output:** `True`
>		Returns whether all items in other, specified set(s) are present in this set	`set1 = {1, 2, 3}; set2 = {2, 3}; print(set1 > set2)` **Output:** `True`
pop()		Removes an element from the set (randomly)	`my_set = {1, 2, 3}; popped_element = my_set.pop(); print(popped_element, my_set)` **Output:** `1 {2, 3}`
remove()		Removes the specified element (raises an error if the element is not found)	`my_set = {1, 2, 3}; my_set.remove(2); print(my_set)` **Output:** `{1, 3}`

symmetric_difference()	^	Returns a set with the symmetric differences of two sets	`set1 = {1, 2, 3}; set2 = {3, 4, 5}; print(set1.symmetric_difference(set2))` **Output:** `{1, 2, 4, 5}`
symmetric_difference_update()	^=	Inserts the symmetric differences from this set and another	`set1 = {1, 2, 3}; set2 = {3, 4, 5}; set1.symmetric_difference_update(set2); print(set1)` **Output:** `{1, 2, 4, 5}`

▼ ***Python Dictionary ?***

→ Dictionaries are used to **store data values in key:value pairs**.

→ A dictionary is a collection which is **ordered*, changeable and do not allow duplicates**.

Dictionary Items :

```
student = {
   "name":"sanjai",
   "age":22,
   "dept":"IT",
   "marks":[89, 76, 99]
}

print(student["name"])
print(student["age"])
print(student["dept"])
print(student["marks"])

# Output
sanjai
22
IT
[89, 76, 99]
```

Dictionary Length :

→ To determine how many items a dictionary has, use the `len()` function:

```
student = {
   "name":"sanjai",
   "age":22,
```

```
   "dept":"IT",
   "marks":[89, 76, 99]
}

# Output
4
```

The dict() Constructor :

→ It is also possible to use the dict() constructor to make a dictionary.

```
student = dict(name="sanjai", age=22, dept="IT")
print(student)

# Output
{'name': 'sanjai', 'age': 22, 'dept': 'IT'}
```

▼ ***Access Items :***

→ You can access the items of a dictionary by referring to its key name, inside square brackets.

→ There is also a method called `get()` that will give you the same result.

```
student = {
   "name":"sanjai",
   "age":22,
   "dept":"IT",
   "marks":[89, 76, 99]
}

print(student["name"])
print(student.get("name"))

# Output
sanjai
sanjai
```

Get Keys :

→ The `keys()` method will return a list of all the keys in the dictionary.

Get Values :

→ The `values()` method will return a list of all the values in the dictionary.

Get Items :

→ The `items()` method will return each item in a dictionary, as tuples in a list.

```
student = {
   "name":"sanjai",
   "age":22,
```

```
   "dept":"IT",
   "marks":[89, 76, 99]
}

print(student.keys())

print(student.values())

print(student.items())

# Output
dict_keys(['name', 'age', 'dept', 'marks'])
dict_values(['sanjai', 22, 'IT', [89, 76, 99]])
dict_items([('name', 'sanjai'), ('age', 22), ('dept', 'IT'), ('marks', [89, 76, 99])])
```

▼ ***Change Items :***

Change Values :

→ You can change the value of a specific item by referring to its key name.

Update Dictionary :

→ The `update()` method will update the dictionary with the items from the given argument.

```
student = {
   "name":"sanjai",
   "age":22,
   "dept":"IT",
   "marks":[89, 76, 99]
}

student["age"] = 25
print(student)

student.update({"age":25})     # using update()
print(student)

# Output
{'name': 'sanjai', 'age': 25, 'dept': 'IT', 'marks': [89, 76, 99]}
{'name': 'sanjai', 'age': 25, 'dept': 'IT', 'marks': [89, 76, 99]}
```

▼ ***Remove Items :***

→ The `pop()` method removes the item with the specified key name.

→ The
`popitem()` method removes the last inserted item.

→ The
`del` keyword removes the item with the specified key name.

→ The
`clear()` method empties the dictionary.

```
student = {
   "name":"sanjai",
   "age":22,
   "dept":"IT",
   "marks":[89, 76, 99]
}

student.pop("age")
print(student)

student = {
   "name":"sanjai",
   "age":22,
   "dept":"IT",
   "marks":[89, 76, 99]
}

student.popitem()
print(student)

student = {
   "name":"sanjai",
   "age":22,
   "dept":"IT",
   "marks":[89, 76, 99]
}

del student["dept"]
print(student)

student = {
   "name":"sanjai",
   "age":22,
   "dept":"IT",
   "marks":[89, 76, 99]
}

student.clear()
print(student)
```

```
# Output
{'name': 'sanjai', 'dept': 'IT', 'marks': [89, 76, 99]}
{'name': 'sanjai', 'age': 22, 'dept': 'IT'}
{'name': 'sanjai', 'age': 22, 'marks': [89, 76, 99]}
{}
```

▼ ***Loop Items :***

Loop Through a Dictionary :

→ You can loop through a dictionary by using a `for` loop.

→ When looping through a dictionary, the return value are the *keys* of the dictionary, but there are methods to return the *values* as well.

```
student = {
    "name":"sanjai",
    "age":22,
    "dept":"IT",
    "marks":[89, 76, 99]
}

for x in student:
    print(x)

for x in student.values():
    print(x)

for x in student.keys():
    print(x)

for x in student.items():
    print(x)

# Output
name
age
dept
marks
sanjai
22
IT
[89, 76, 99]
name
age
dept
marks
('name', 'sanjai')
('age', 22)
('dept', 'IT')
('marks', [89, 76, 99])
```

▼ ***Dictionary Methods :***

Method	Description	Example
clear()	Removes all the elements from the dictionary	`d = {'a': 1, 'b': 2}` `d.clear()` `print(d)` ⇒ `{}`
copy()	Returns a copy of the dictionary	`d = {'a': 1, 'b': 2}` `new_d = d.copy()` `print(new_d)` ⇒ `{'a': 1, 'b': 2}`
fromkeys()	Returns a dictionary with the specified keys and value	`d = dict.fromkeys(['a', 'b'], 0)` `print(d)` ⇒ `{'a': 0, 'b': 0}`
get()	Returns the value of the specified key	`d = {'a': 1, 'b': 2}` `print(d.get('a'))` ⇒ `1`
items()	Returns a list containing a tuple for each key-value pair	`d = {'a': 1, 'b': 2}` `print(d.items())` ⇒ `[('a', 1), ('b', 2)]`
keys()	Returns a list containing the dictionary's keys	`d = {'a': 1, 'b': 2}` `print(d.keys())` ⇒ `dict_keys(['a', 'b'])`
pop()	Removes the element with the specified key	`d = {'a': 1, 'b': 2}` `d.pop('a')` `print(d)` ⇒ `{'b': 2}`
popitem()	Removes the last inserted key-value pair	`d = {'a': 1, 'b': 2}` `d.popitem()` `print(d)` ⇒ `{'a': 1}`
setdefault()	Returns the value of the specified key, inserts the key with a value if not found	`d = {'a': 1}` `d.setdefault('b', 2)` `print(d)` ⇒ `{'a': 1, 'b': 2}`
update()	Updates the dictionary with the specified key-value pairs	`d = {'a': 1}` `d.update({'b': 2})` `print(d)` ⇒ `{'a': 1, 'b': 2}`
values()	Returns a list of all the values in the dictionary	`d = {'a': 1, 'b': 2}` `print(d.values())` ⇒ `dict_values([1, 2])`

▼ ***Python if...Else ?***

Python Conditions and If statements :

→ Python supports the usual logical conditions from mathematics:

1. **Equals: a == b**
2. **Not Equals: a != b**
3. **Less than: a < b**
4. **Less than or equal to: a <= b**
5. **Greater than: a > b**
6. **Greater than or equal to: a >= b**

→ These conditions can be used in several ways, most commonly in "if statements" and loops.

→ An "if statement" is written by using the if keyword.

```
x = 50
y = 100

if y > x:
  print("y is greater than x")

# Output
y is greater than x
```

Elif :

→ The elif keyword is Python's way of saying "if the previous conditions were not true, then try this condition".

```
x = 50
y = 100
if y > x:
  print("y is greater than x")
elif x == y:
  print("x and y are equal")

# Output
y is greater than x
```

Else :

→ The else keyword catches anything which isn't caught by the preceding conditions.

```
x = 50
y = 100
if y > x:
  print("y is greater than x")
elif x == y:
  print("x and y are equal")
else:
  print("x is greater than y")

# Output
y is greater than x
```

Ternary Operators :

```
x = 50
y = 100
print("X") if x > y else print("=") if x == y else print("Y")

# Output
Y
```

And :

→ The and keyword is a logical operator, and is used to combine conditional statements:

Or :

→ The `or` keyword is a logical operator, and is used to combine conditional statements:

Not :

→ The `not` keyword is a logical operator, and is used to reverse the result of the conditional statement:

Nested If :

→ You can have `if` statements inside `if` statements, this is called *nested* `if` statements.

```
x = 200
y = 33
z = 500
if x > y and z > x:
  print("Both conditions are True")


x = 200
y = 33
z = 500
if x > y or x > z:
  print("Both conditions are True")


x = 33
y = 200
if not x > y:
  print("x is NOT greater than y")


x = 50

if x > 10:
  print("Above ten,")
  if x > 20:
    print("and also above 20!")
  else:
    print("but not above 20.")

# Output
Both conditions are True
Both conditions are True
x is NOT greater than y
Above ten,
and also above 20!
```

▼ ***While Loop ?***

The while Loop :

→ With the while loop we can execute a set of statements as long as a condition is true.

The break Statement :

→ With the break statement we can stop the loop even if the while condition is true.

The continue Statement :

→ With the continue statement we can stop the current iteration, and continue with the next.

The else Statement :

→ With the else statement we can run a block of code once when the condition no longer is true.

```
# While Loop

x = 0

while x < 11:
    print(x)
    x = x + 1

# Output
0
1
2
```

```
3
4
5
6
7
8
9
10

-------------------------------------------------------------------------------------------------------------------

# While Loop with break statement

x = 1

while x < 11:
    print(x)
    if x == 5:
        break
    x = x + 1

# Output
1
2
3
4
5

-------------------------------------------------------------------------------------------------------------------

# While Loop with continur Statement

while x < 10:
    x = x + 1
    if x == 5:
        continue
    print(x)

# Output
1
2
3
4
6
7
8
9
10

-------------------------------------------------------------------------------------------------------------------

# While Loop With Else Block

x = 1
while x < 6:
  print(x)
  x = x + 1
else:
```

```
  print("x is no longer less than 6")

# Output
1
2
3
4
5
x is no longer less than 6
```